

CAT User and Administrator Manual

A shared human and AI workspace for discussing a codebase

Paul Fishwick and Claude Code

September 2026

Contents

1	Introduction	2
1.1	Design goals	2
1.2	What CAT is not	3
2	Architecture	3
2.1	Why the browser can hold the keys it does	4
2.2	Why the agent answers asynchronously	4
3	The permission model	4
3.1	A subtlety worth stating	4
4	Data model	5
4.1	Specialties and code areas	5
4.2	Messages	5
5	Using CAT	6
5.1	Signing in	6
5.2	Channels	6
5.3	Talking to an agent	6
5.4	Interface size	6
6	Administration	6
6.1	Adding a person	6
6.2	Seeing who is in the workspace	7
6.3	Adding a channel	7
6.4	Adding an agent	7
7	Deployment	7
7.1	Supabase	7
7.2	The Edge Function	8
7.3	GitHub Pages	8
8	Testing	8
9	Limitations and future work	8

1 Introduction

CAT is a small chat workspace in which a research group discusses a specific GitHub codebase, with AI agents participating as named members of the conversation. It resembles Slack in shape—a sidebar of channels, a message list, a composer—but it is deliberately much smaller, and it makes one commitment Slack does not: each channel is bound to a repository, and the agents in that channel can read it.

The system is designed for a group of fewer than twenty people. That constraint is not incidental. It is what makes it reasonable to let every member read every channel, to manage membership by adding rows to a table, and to run the whole thing on free hosting.

A second constraint shapes it just as much: only some people may invoke an AI agent. Everyone can read, everyone can talk, but summoning an agent spends money against an API key, and that permission is granted individually. Hiding a button would not be a control, so the check lives on the server, in the one place the key exists.



Figure 1: The CAT workspace. Channels and agents are listed in the sidebar, with people grouped by discipline beneath them. The badge under the member’s own name indicates permission to invoke agents.

1.1 Design goals

- **One channel per codebase.** #multimodel is about github.com/metaphorz/Multimodel. Adding a repository means adding a row, not deploying anything.
- **Agents are participants, not features.** They appear in the roster, they are addressed by name, and their replies are ordinary messages in the transcript.
- **An interdisciplinary group is the normal case.** Members carry a discipline, and the agent is told who is asking, so a statistician and a meteorologist asking the same question get different answers.
- **Permissions are enforced where they cannot be bypassed.**

- **No build step.** The client is three static files. It is deployed by copying them to GitHub Pages.

1.2 What CAT is not

The agents read and discuss. They cannot write files, push branches, or open pull requests—there is no code path by which they could. Having an agent propose real changes through a GitHub Action is a plausible next phase, but it is a different trust decision and deserves to be made deliberately rather than inherited.

There are also no threads, reactions, file uploads, or message editing. Recreating Slack is not the goal; having somewhere to think about a codebase together is.

2 Architecture

CAT is a static web page and a managed backend. There is no server to administer and nothing to keep running.

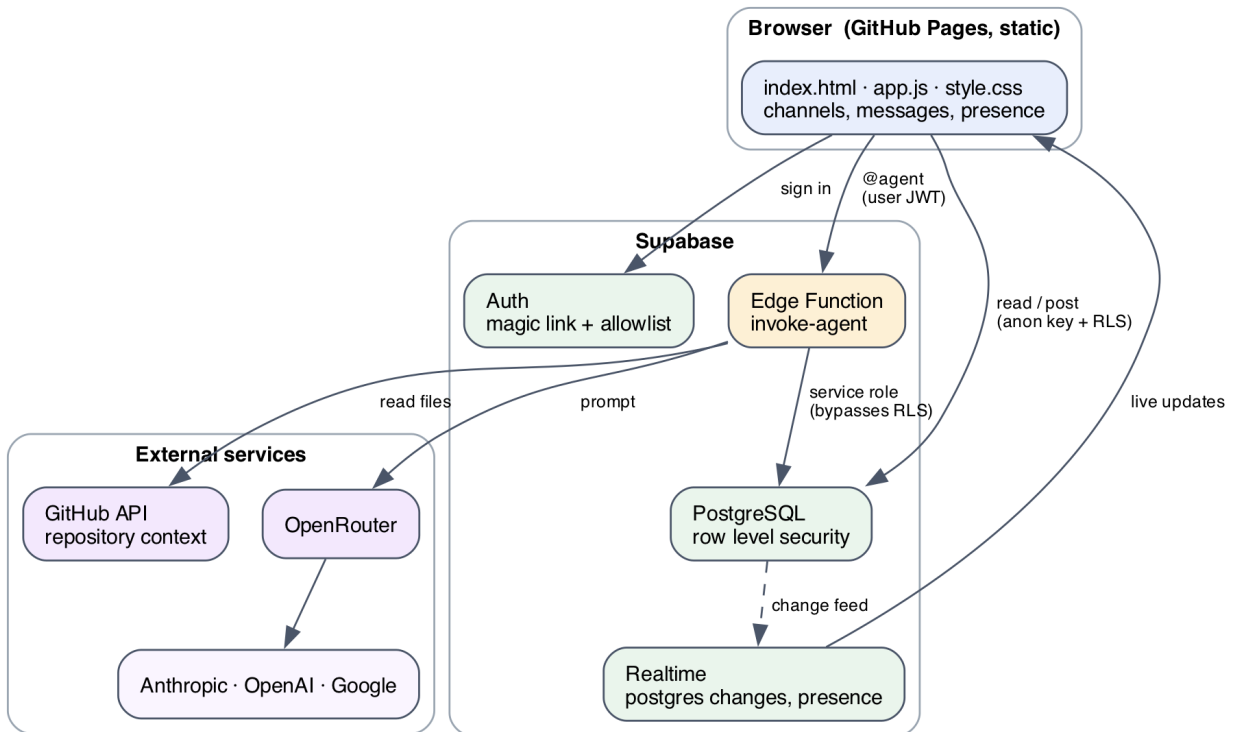


Figure 2: The whole system. The browser holds no secrets; the Edge Function holds all of them.

GitHub Pages serves `index.html`, `app.js`, `style.css` and `config.js`. These are static files with no build step, published by a GitHub Actions workflow on every push.

Supabase provides authentication, a PostgreSQL database with row level security, a realtime change feed, and the Edge Function runtime. The free tier is far larger than this workload requires.

The Edge Function (`invoke-agent`) is the security boundary. It identifies the caller, decides whether they may summon an agent, and only then reaches for the API key.

OpenRouter reaches Anthropic, OpenAI and Google models through a single credential, which is what makes adding an agent a database row rather than a code change.

2.1 Why the browser can hold the keys it does

`config.js` contains the Supabase project URL and the publishable (anonymous) key, and both are visible to anyone who views source. This is intended. The anonymous key is an identifier, not an authorisation: every table is protected by row level security, and an unauthenticated caller holding that key can read nothing at all. The credentials that matter—the OpenRouter key, the GitHub token, the service role key—exist only as Edge Function secrets and never reach a browser.

2.2 Why the agent answers asynchronously

A model call on a real question can take longer than a browser will keep a request open. Rather than hold the connection, `invoke-agent` inserts an empty message with `status = 'pending'`, returns immediately, and finishes the work in the background, updating that row when the answer arrives.

The realtime feed turns this into a feature. Every connected client sees “Claude is thinking” the moment the agent is summoned, and sees the reply replace it when it lands—without polling and without reloading.

3 The permission model

Three mechanisms hold the system up, and it is worth knowing which does what, because they fail differently.

Getting in at all is the `allowlist` table together with a trigger on user creation. An address that is not listed cannot create an account: the trigger raises an exception and the signup fails. This matters because the page is public—anyone can load the sign-in screen.

Reading and posting is row level security. A member may insert messages as themselves and as nobody else, and messages carrying an `agent_slug` are rejected outright, so a collaborator cannot forge words from an agent. Agent messages are written only by the service role, which the browser never possesses.

Invoking an agent is the Edge Function checking `can_invoke_agent` before it touches the API key.

3.1 A subtlety worth stating

Row level security decides *which rows* a member may change; it cannot decide *which columns*. Members are allowed to edit their own profile row, and without a further restriction that would let anyone grant themselves `can_invoke_agent`. The schema therefore also issues a column-level grant:

	Read	Post	Invoke agents
Paul	yes	yes	yes
Other members	yes	yes	no
Agents	yes	yes	—
Anyone else	no	no	no

Table 1: The default arrangement. Permission to invoke agents is granted per person in the **members** table.

```
revoke update on public.members from authenticated;
grant update (display_name, github_login, username, specialty)
on public.members to authenticated;
```

`can_invoke_agent` and `is_admin` are deliberately absent from that list.

4 Data model

Seven tables, shown in Figure 5.

The distinction between **allowlist** and **members** is the one that confuses people. **allowlist** is an invitation: an address, and what that person will be allowed to do. **members** is created automatically the first time they actually sign in. Someone invited but absent exists in the first table and not the second, and is therefore invisible in the interface.

4.1 Specialties and code areas

Each member carries a **specialty**—meteorologist, statistician, actuary, geospatial analyst, and so on. Each channel additionally maps specialties to real paths in *that* repository, through **channel_areas**. The mapping is per channel rather than global because “the statistics code” means something different in every repository.

The agent receives both. When a statistician asks how the wind field is generated, it can answer in the vocabulary of sampling and variance and point at `pipeline/fit_metamodels.py`; when a meteorologist asks the same question it can talk about surface roughness and point at `pipeline/windfield.py`. When a question lands squarely in someone else’s discipline, it is instructed to say so and name that person rather than answer at the edge of its competence.

4.2 Messages

A message is written either by a person or by an agent, never both, and a check constraint enforces it:

```
constraint messages_speaker_check check (
    (author_id is not null and agent_slug is null) or
    (author_id is null and agent_slug is not null)
)
```

The **status** column carries **pending**, **complete** or **error**, which is how the “thinking” state is represented. The **metadata** column records, for agent messages, which model answered, who asked, and the token usage—visible in small type beneath each reply.

5 Using CAT

5.1 Signing in

There is no password. Enter an allowlisted email address, and a single-use link arrives by mail.

Two things commonly go wrong. The link must be opened on a machine that can reach the address it redirects to, and it must be opened in the browser you intend to use—a session created in one browser does not exist in another.

5.2 Channels

Each channel corresponds to a repository, named for it, and the header links directly to it on GitHub. Selecting a channel loads its history and subscribes to live updates; the channel is reflected in the page URL, so a particular channel can be linked to or bookmarked.

5.3 Talking to an agent

Mention an agent by name anywhere in a message:

```
@claude what does this codebase model, and where should a
statistician look first?
```

The composer indicates whether the mention will summon anyone. Members without permission may still write `@claude`—the text is not forbidden—but nothing will answer, and the composer says so plainly rather than failing silently.

The agent is given the recent conversation, the roster with everyone’s discipline, the specialty-to-path map for that channel, and the repository’s overview document and source file listing. It is told who asked and what their discipline is.

5.4 Interface size

The sidebar carries a size control offering five steps. It scales the entire interface—type, avatars, sidebar width—rather than type alone, because a large font inside a fixed-width column reads worse than no change at all. The choice is remembered per browser.

6 Administration

6.1 Adding a person

One row, in the Supabase SQL editor:

```
insert into public.allowlist
  (email, display_name, username, specialty, can_invoke_agent)
values
  ('alice@example.edu', 'Alice_Reyes', 'alice', 'statistics', false);
```

They then sign in themselves; the member record is created on first login. Setting the specialty is worth doing at this point, since it is what lets the agent pitch its answers correctly.

To change an existing member’s permissions, update `members`—the `allowlist` value only seeds the initial state:

```
update public.members set can_invoke_agent = true
where username = 'alice';
```

6.2 Seeing who is in the workspace

Three states are easy to confuse: *invited* (on the allowlist, possibly never seen), *member* (has signed in at least once), and *present* (has the application open right now). Only the last two appear in the sidebar, and presence is live state that is not stored anywhere.

```
select a.email,
       coalesce(m.display_name, a.display_name) as name,
       m.username,
       (m.id is not null) as has_signed_in,
       u.last_sign_in_at
from public.allowlist a
left join public.members m on lower(m.email) = lower(a.email)
left join auth.users u on u.id = m.id
order by u.last_sign_in_at desc nulls last, a.email;
```

6.3 Adding a channel

```
insert into public.channels
(slug, name, purpose, github_owner, github_repo, position)
values
('hurricane', 'hurricane', 'Storm_□track_□generation.',
 'metaphor', 'Hurricane', 1);
```

If the repository is private, the configured GitHub token needs read access to it as well. Consider also populating `channel_areas` for the new channel, since that is what lets the agent direct people to the right files.

6.4 Adding an agent

Because the Edge Function dispatches on `agents.provider`, adding a model requires no deployment at all:

```
insert into public.agents (slug, display_name, provider, model)
values ('gemini', 'Gemini', 'openrouter',
       'google/gemini-3.1-pro-preview');
```

Adding a new *provider*—something other than `anthropic` or `openrouter`—does require a branch in the function and a redeployment.

Note that adding agents widens what can be asked for; it does not widen who may ask.

7 Deployment

7.1 Supabase

Create a project, then run the migrations in `supabase/migrations/` in order, in the SQL editor. Under **Authentication** → **URL Configuration**, set the site URL and the redirect URLs, including both the published address and `http://localhost:8000/` for local work. Note the trailing slashes: the client asks to be returned to `origin + pathname`, which always ends in one, and an entry lacking it will not match.

The built-in email sender is limited to roughly two messages per hour and is intended for testing. Configure custom SMTP before inviting anyone.

7.2 The Edge Function

```
supabase secrets set --project-ref <ref> OPENROUTER_API_KEY=...
supabase secrets set --project-ref <ref> GITHUB_TOKEN=...
supabase functions deploy invoke-agent --project-ref <ref>
```

SUPABASE_URL, SUPABASE_ANON_KEY and SUPABASE_SERVICE_ROLE_KEY are injected automatically and must not be set by hand.

The GitHub token is optional. Without it the agent can still discuss the conversation, but a private repository is invisible to it; it is told so, and says so, rather than guessing.

7.3 GitHub Pages

Pages must be served from a public repository unless the account has a paid plan. This is safe: no credential lives in any committed file. A public page is not public access—anyone may load the sign-in box, and the allowlist decides everything after that.

8 Testing

tests/test_screens.py drives a real browser through every screen using Selenium: the sign-in gate, the refusal of an address that is not on the allowlist, the signed-in workspace, posting, code-block rendering, escaping of markup in messages, and the agent hint in the composer. Screenshots are written to tests/screenshots/.

Two aspects are worth noting. Magic-link authentication leaves nothing to type, so the suite mints a sign-in link through Supabase’s admin API and drives the browser to it. And because there is only one database, everything the suite writes is tagged and deleted afterwards.

```
export SUPABASE_SERVICE_ROLE_KEY=...
python3 tests/test_screens.py -v # all but the agent
CAT_TEST_AGENT=1 python3 tests/test_screens.py -v # including it
```

Invoking an agent costs money, so that test is opt-in. By default the suite checks that the composer offers the agent, not that a vendor answers.

9 Limitations and future work

- **Agents cannot change code.** Dispatching a GitHub Action that opens a pull request is the obvious next phase, and the schema already reserves `metadata` for recording the resulting branch or PR.
- **Every member sees every channel.** Reasonable below twenty people; it would need per-channel membership beyond that.
- **Repository context is a file listing and an overview document,** not the code itself. The agent knows what exists and can reason about structure, but cannot read a function body it was not shown.
- **Free-tier projects pause after a week of inactivity** and are resumed from the dashboard with data intact.

- **Presence is not history.** There is no record of who was here yesterday, only who is here now and when each person last signed in.

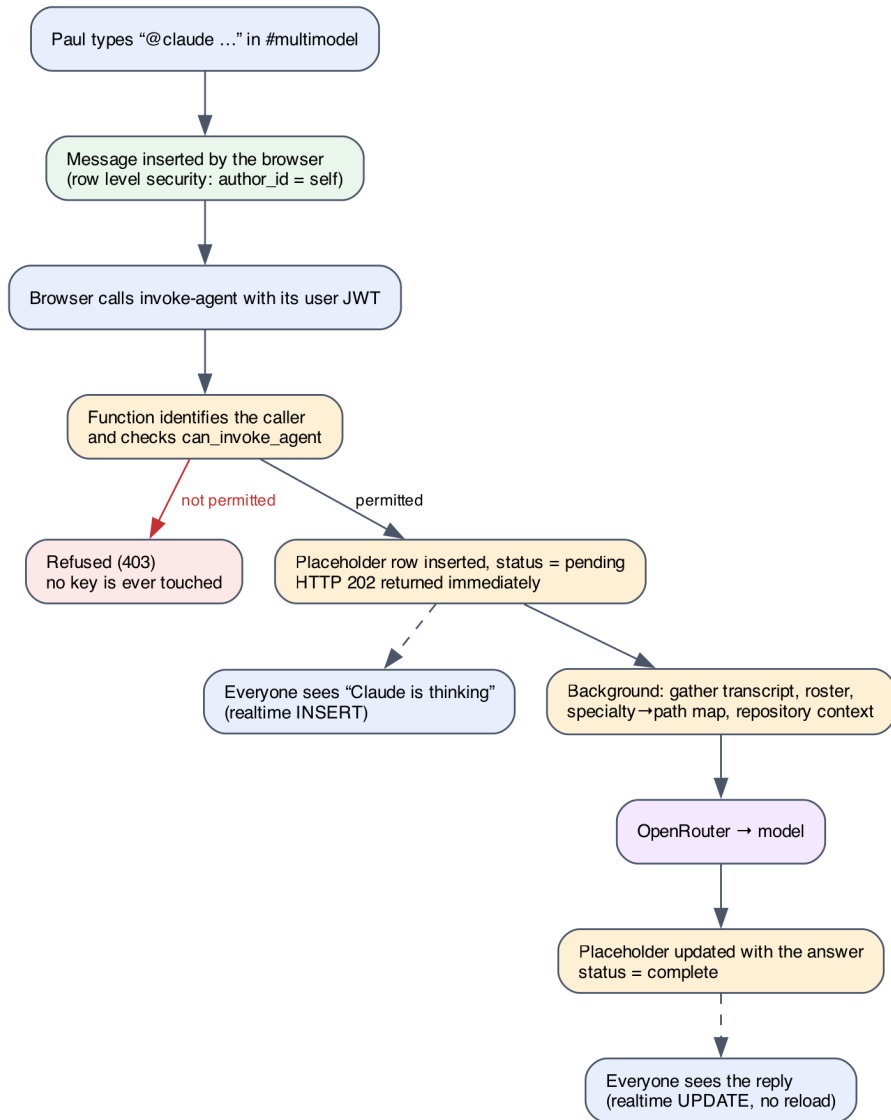


Figure 3: Invoking an agent. The permission check happens before any credential is touched, and the placeholder makes the wait visible to everyone.

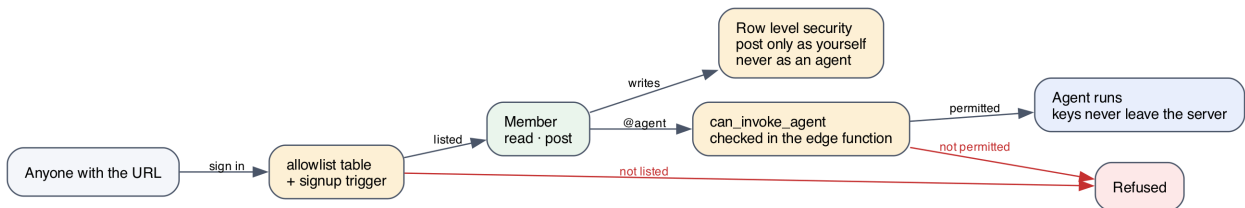


Figure 4: Where each decision is made. Nothing in the browser is trusted.

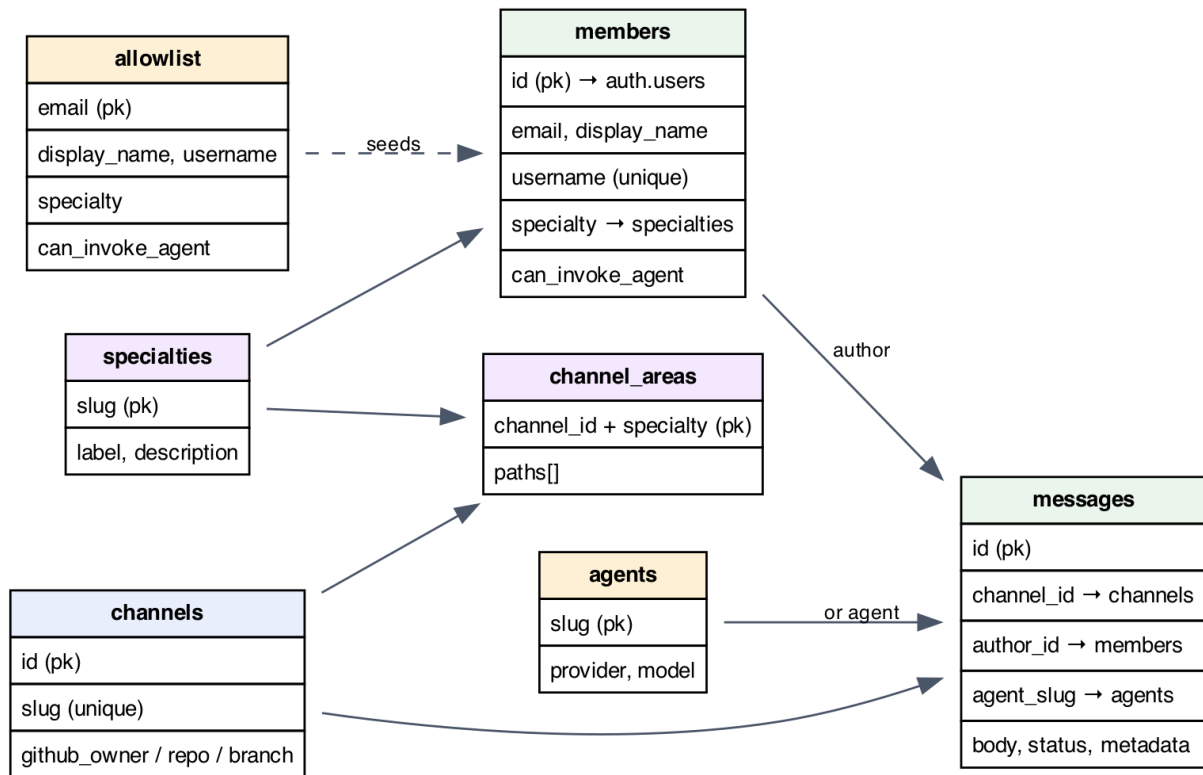


Figure 5: The schema. `allowlist` governs who may exist; `members` records who does.

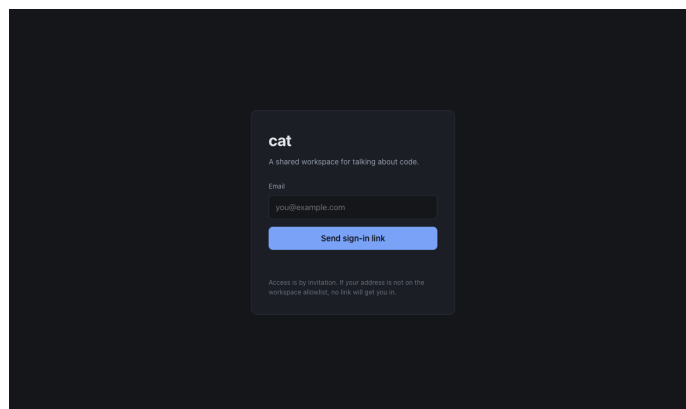


Figure 6: The sign-in screen. It is all anyone sees without an invitation.